Object Oriented Programming

Chapter 4:

Composition

Objectives

At the end of this Lecture, students will become familiar with:

- **Composition and how to use it in C#;**
- **The difference between Composition and Aggregation;**
- **Nested classes and how to define it;**
- **Partial classes and how to use it.**

Outline

- Composition
- Aggregation
- Nested Classes
- Partial Classes

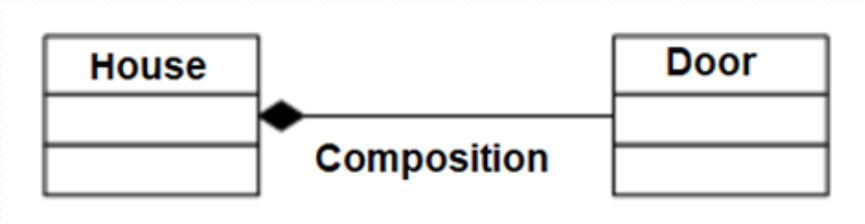
Composition

- The reusability principle has led to the emergence of many relationships between classes; among these relationships are: composition, aggregation, nesting, and partitioning;
- When a new class is written, a new Data Type is created; it can be used to create objects from it. For example, when writing the class **Door**, it can be used within the class **House** as one of its members. We call the relationship between these two classes **Composition**, because every object from the class **House** will have among its members an object from the class **Door**.
- The composition relationship between the two classes is represented by "**composed-of**";
- The concept of "Composition" could be expressed as the class defined by a developer could be used as a data type for members of another class.

```
class Door {
//....
}

class House {
Door door = new Door();
//.....
}
```

Composition



- When you create an object from **House**, a **Door** object will be created as a part of it. As long as the first object is in the memory, the second object will also be present.
- When you destroy the first object, the second will be destroyed because it is one of its components
- If we create the class **Date** and the class **Employee**, the class **Employee** can contain more than one member of the **Date** datatype, such as **BirthDate** for the date of birth and **HireDate** for the date of employment.
- When the instance of Employee is destroyed, both objects
- copied from the class Date will be destroyed.

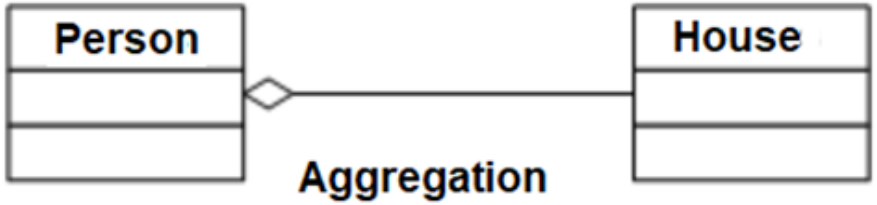
```
class Date {
//....
}

class Employee {
 //....
    Date BirthDate = new Date();
    Date HireDate = new Date();
//....
}
```

Aggregation

- Although they are sometimes used as equivalent concepts, we must distinguish between **composition** and **aggregation**.

```
public class House {
        //....
        }
public class Person {
              private House house;
              public Person(House house)
              {
                  this.house = house;
         }
              //....
        }
```



The aggregation is represented by the relationship "**has-a**", so if we create the class **Person** that owns an object from the class **House** as shown aside, and use the two classes as follows:

```
House house = new House();

Person person = new Person(house);
```

- Then, the **house** object will be created from the class **House** and the **person** object will be created from the class **Person** that owns the **house** object. Destroying the **person** object will not destroy the **house**. Another object can be created from the class **Person** that owns the same **house**.

Nested Classes

- A class can be created within another class as one of its members, and it is called **Nested Class**. In this case, the members of the nested class can access all the members of the container class whatever the access specifier is private, protected, public, etc.
- In the following example, the class **Nested** is a member of the class **Container**:

```
public class Container {
    //....
    class Nested     {
        Nested() { }
        //....
    }

}
```

- Like other members of the class **Container**, the class **Nested** can access all the components of **Container**.
- The default access specifier for a nested class is **private**, and it can be replaced by any other access specifier such as **public**, **protected**, **internal**, etc.

Nested Classes

- In the following example, the nested class has a public access specifier and the containing class has a public access specifier as well:

```
1. public class Container {
2.     public class Nested {
3.         Nested() { }
4.     }
}
```

- To create an object from the nested class, we use its full name. So, to create the **nest** object from the class **Nested**, we should write:

```
Container.Nested nest = new Container.Nested();
```

Nested Classes

- The nested class can access the elements of the containing class by defining a constructor who takes as input an object from the containing class; this is done as follows:

```
1. public class Container {
2.     public class Nested  {
3.
4.         private Container parent;
5.
6.         public Nested( ) {
7.         }// end default constructor
8.
9.         public Nested(Container parent) {
10.             this.parent = parent;
11.
12.         }// end constructor with one object of Container class
13.     }// end class Nested
  }// end class Container
```

Partial Classes

- In some cases, we need to split a class into several **Partial Classes**, such as when the class size is very large. So, a group of programmers can work on it at the same time.
- A class is split into Partial Classes using the word **Partial** with each part, as shown in the following example:

```
public partial class Employee {
    public void DoWork() {   }
       }

public partial class Employee {
    public void GoToLunch()  {  }

      }
```

- All parts must be available during compilation phase, they must all have the same access specifier, and the word **Partial** must come immediately before the class word.

Partial Classes

- If one of the partial classes is declared as **abstract**, the whole class is treated as an abstract class, meaning that all the parts become abstract. Likewise, if one part is declared **sealed**, the entire class is treated as a sealed class.
- And when one of the partial classes inherits another class, all parts become its inheritors and the entire class becomes its inheritor.
- When the partial classes inherit different **interfaces**, the entire class becomes the inheritor of all those interfaces. Each **struct** or **interface** defined within a partial class becomes a member that is available to the entire class. We illustrate this idea through the following:

```
partial class Earth : Planet, IRotate { }
partial class Earth : IRevolve { }
```

- The preceding lines are equivalent to the following line:

```
class Earth : Planet, IRotate, IRevolve { }
```

Partial Classes

- A nested class can also be split, whether the containing class is split or not.
- The following instructions show a nested class that is nested within a container class:

The following instructions show the nested class **NestedClass** within the class **ContainerClass**:

```
partial class Container        {
    partial class NestedClass { }
}
partial class Container
{
    partial class NestedClass { }

}
```

```
class Container {
    partial class Nested {
        void Test() { }
    }
    partial class Nested
    {
        void Test2() { }
    }

}
```

Partial Interface and Partial Struct

```
1. using System;
2. namespace PartialStructInterface {
3.     // First part of The interface
4.     partial interface ITest    {
5.         void Interface_Test();
6.     }// end interface ITest 1
7.
8.     // Second part of The interface
9.     partial interface ITest    {
10.         void Interface_Test2();
11.     }// end interface ITest 2
12.     // First part of The struct
13.     partial struct S     {
14.         void Struct_Test() { }
15.     }// end struct S 1
16.     // Second part of The struct
17.     partial struct S     {
18.         void Struct_Test2() { }
19.     }// end struct S 2
}// end namespace PartialStructInterface
```

Note: The **abstract** class, **sealed** class, **interface**, and **struct** will be studied later.

Summary

- Composition
- Aggregation
- Nested Classes
- Partial Classes

Thank you very much for attending

You can kindly ask questions